In [1]:
```
using BifurcationKit, LinearAlgebra, Parameters
using JLD2
#using GLMakie
using ForwardDiff
using Plots
theme(:dark)
using DelimitedFiles
const BK = BifurcationKit
#using ProgressMeter
using JupyterFormatter
enable_autoformat();
```

In [2]:
```
function tf(x)
    return exp.(x) ./ (1.0 .+ exp.(x))
end
```

Out[2]:
```
tf (generic function with 1 method)
```

In [3]:
```
function itf(y)
    return log.(y ./ (1 .- y))
end
```

Out[3]:
```
itf (generic function with 1 method)
```

In [4]:
```
function fe(phi, p)
    @unpack d2, chi, N, nuA, lambda, switch_dh, zplus, zpep = p

    phiplus = phi[1]
    phiminus = phi[2]
    phipep = phi[3]
    phioh = phi[4]
    phiw = phi[5]

    fFH = (
        (phiplus > 0 ? phiplus * log(phiplus) : 0) +
        (phiminus > 0 ? phiminus * log(phiminus) : 0) +
        (phipep > 0 ? phipep * log(phipep) / (N * nuA) : 0) +
        (phioh > 0 ? phioh * log(phioh) : 0) +
        (phiw > 0 ? phiw * log(phiw) : 0) +
        chi * phipep * phiw
    )


    kappa = sqrt(
        lambda * (
            zplus * zplus * abs(phiplus) +
            abs(phiminus) +
            abs(phioh) +
            (zpep[1]^2 + zpep[2]^2) / (N * nuA) * abs(phipep)
        ),
    )

    fDH = -1 / (4 * pi) * (log(1 + kappa) - kappa + kappa^2 / 2)

    return fFH + switch_dh * fDH
end
```

Out[4]: fe (generic function with 1 method)

In [5]:
```
gradfe(x, p) = ForwardDiff.gradient(y -> fe(y, p), x)
```

Out[5]: gradfe (generic function with 1 method)

In [6]:
```
Hfe(x, p) = ForwardDiff.hessian(y -> fe(y, p), x)
```

Out[6]: Hfe (generic function with 1 method)

In [7]:
```
function evmin(y, p)
    @unpack d2, chi, N, nuA, lambda, zplus, zpep = p

    phiplus = y[1]
    phiminus = y[2]
    phipep = y[3]
    phioh = (zpep[1] + zpep[2]) / (N * nuA) * phipep
    phiw = 1 - phiplus - phiminus - phipep - phioh

    x = [phiplus, phiminus, phipep, phioh, phiw]

    Hall = Hfe(x, p)

    G = [
        1 0 0 zplus -(1 + zplus)
        0 1 0 -1 0
        0 0 1 (zpep[1]+zpep[2])/(N*nuA) -(1 + (zpep[1] + zpep[2]) / (N *
    ]

    H = G * Hall * G'

    ev = eigvals(H)

    return ev[argmin(abs.(ev))]
end
```

Out[7]: evmin (generic function with 1 method)

In [8]:
```
function F_dhlen(x, p)
    f = similar(x)
    n = length(x)

    @unpack d2, chi, N, nuA, lambda, zplus, KM, zpep = p

    phi1_1 = tf(x[1])
    phi2_1 = tf(x[2])
    phi3_1 = tf(x[3])
    phi4_1 = tf(x[4])
    phi5_1 = tf(x[5])

    phi1_2 = tf(x[6])
    phi2_2 = tf(x[7])
    phi3_2 = tf(x[8])
    phi4_2 = tf(x[9])
    phi5_2 = tf(x[10])

    lamnv1 = x[11]
    lamnv2 = x[12]
```

```
    lamen1 = x[13]
    lamen2 = x[14]

    om1 = x[15]
    om2 = x[16]
    om3 = x[17]


    fe_1 = fe([phi1_1, phi2_1, phi3_1, phi4_1, phi5_1], p)
    fe_2 = fe([phi1_2, phi2_2, phi3_2, phi4_2, phi5_2], p)

    gfe1_1, gfe2_1, gfe3_1, gfe4_1, gfe5_1 =
        gradfe([phi1_1, phi2_1, phi3_1, phi4_1, phi5_1], p)
    gfe1_2, gfe2_2, gfe3_2, gfe4_2, gfe5_2 =
        gradfe([phi1_2, phi2_2, phi3_2, phi4_2, phi5_2], p)

    a = [zplus, -1, (zpep[1] + zpep[2]) / (N * nuA), -1, 0]

    f[1] = gfe1_1 + lamnv1 + lamen1 * a[1] + om1
    f[2] = gfe2_1 + lamnv1 + lamen1 * a[2] + om2
    f[3] = gfe3_1 + lamnv1 + lamen1 * a[3] + om3
    f[4] = gfe4_1 + lamnv1 + lamen1 * a[4]
    f[5] = gfe5_1 + lamnv1 + lamen1 * a[5]

    f[5+1] = gfe1_2 + lamnv2 + lamen2 * a[1] + om1
    f[5+2] = gfe2_2 + lamnv2 + lamen2 * a[2] + om2
    f[5+3] = gfe3_2 + lamnv2 + lamen2 * a[3] + om3
    f[5+4] = gfe4_2 + lamnv2 + lamen2 * a[4]
    f[5+5] = gfe5_2 + lamnv2 + lamen2 * a[5]

    f[11] =
        fe_1 - fe_2 +
        om1 * (phi1_1 - phi1_2) +
        om2 * (phi2_1 - phi2_2) +
        om3 * (phi3_1 - phi3_2)

    f[12] = phi1_1 + phi2_1 + phi3_1 + phi4_1 + phi5_1 - 1
    f[13] = phi1_2 + phi2_2 + phi3_2 + phi4_2 + phi5_2 - 1
    f[14] = a[1] * phi1_1 + a[2] * phi2_1 + a[3] * phi3_1 + a[4] * phi4_1
    f[15] = a[1] * phi1_2 + a[2] * phi2_2 + a[3] * phi3_2 + a[4] * phi4_2

    f[16] = phi3_1 - KM
    f[17] = zplus * phi1_1 - phi2_1

    return f

end
```

Out[8]: F_dhlen (generic function with 1 method)

In [9]:

```
function build_run(case)

    phi1_1 = case.phivar1[2]
    phi2_1 = case.phivar1[3]
    phi3_1 = case.phivar1[1]
    phi4_1 =
        case.zplus * phi1_1 - phi2_1 +
        (case.zpep[1] + case.zpep[2]) / (case.N * case.nuA) * phi3_1
    phi5_1 = 1 - phi1_1 - phi2_1 - phi3_1 - phi4_1
```

```
    phi1_2 = case.phivar2[2]
    phi2_2 = case.phivar2[3]
    phi3_2 = case.phivar2[1]
    phi4_2 =
        case.zplus * phi1_2 - phi2_2 +
        (case.zpep[1] + case.zpep[2]) / (case.N * case.nuA) * phi3_2
    phi5_2 = 1 - phi1_2 - phi2_2 - phi3_2 - phi4_2

    dd2 = (phi1_1 - phi1_2)^2 + (phi2_1 - phi2_2)^2 + (phi3_1 - phi3_2)^2

    par = (
        d2 = dd2,
        chi = case.chi,
        N = case.N,
        nuA = case.nuA,
        lambda = case.lambda,
        KM = case.KM,
        switch_dh = case.switch_dh,
        zplus = case.zplus,
        zpep = case.zpep,
        p_max = case.p_max,
        p_min = case.p_min,
    )

    gfe1_1, gfe2_1, gfe3_1, gfe4_1, gfe5_1 =
        gradfe([phi1_1, phi2_1, phi3_1, phi4_1, phi5_1], par)
    gfe1_2, gfe2_2, gfe3_2, gfe4_2, gfe5_2 =
        gradfe([phi1_2, phi2_2, phi3_2, phi4_2, phi5_2], par)

    #    a = [zplus , -1, 2 / (N * nuA), -1, 0]


    lamnv1 = -gfe5_1
    lamnv2 = -gfe5_2
    lamen1 = lamnv1 + gfe4_1
    lamen2 = lamnv2 + gfe4_2
    om1 = -(gfe1_1 + lamnv1 + lamen1 * case.zplus)
    om2 = -(gfe2_1 + lamnv1 + lamen1 * (-1))
    # CHECK!!
    om3 = -(gfe3_1 + lamnv1 + lamen1 * (case.zpep[1] + case.zpep[2]) / (c

    sol0 = [
        itf(phi1_1),
        itf(phi2_1),
        itf(phi3_1),
        itf(phi4_1),
        itf(phi5_1),
        itf(phi1_2),
        itf(phi2_2),
        itf(phi3_2),
        itf(phi4_2),
        itf(phi5_2),
        lamnv1,
        lamnv2,
        lamen1,
        lamen2,
        om1,
        om2,
        om3,
    ]
```

```
    return (sol0 = sol0, par = par, phivar1 = case.phivar1, phivar2 = cas
end
```

Out[9]: build_run (generic function with 1 method)

In [10]:
```
# Case 1 should correspond to the violet line on fig 1(a) in my and Giuli
case1 = (
    chi = 0.85,
    N = 7,
    nuA = 50 / 7,
    lambda = 100.0,
    #    KM = 1.35e-3,
    KM = 1.5 * 1e-3 * 50 / 55.35,
    switch_dh = 1,
    zplus = 1.0,
    zpep = [1, 1],
    p_max = 125.0,
    p_min = 0.1,
    phivar1 = (
        0.0014072188552725978,
        0.013975575489771453,
        0.013975575489771453,
        5.420054200542007e-5,
        0,
        1.9049897537435612,
    ),
    phivar2 = (
        0.4713311048662077,
        0.00752500303794388,
        0.026272431023913592,
        9.901500213468598e-5,
        0.5717491375633551,
        2.5747871316756075,
    ),
);

# Case 2 ROUGHLY corresponds to the violet line on fig 1(b) in my and Giu
case2 = (
    chi = 0.8555,
    N = 7,
    nuA = 50 / 7,
    lambda = 120.0,
    #    KM = 1.35e-3,
    KM = 1.5 * 1e-3 * 50 / 55.35,
    switch_dh = 1,
    zplus = 2.0,
    zpep = [1, 1],
    p_max = 124.0,
    p_min = 0.1,
    phivar1 = (
        0.0013550135501355016,
        1.3089264695931858e-5,
        2.6178528880041418e-5,
        6.855740370939678e-5,
        0,
        0.1410653171572616,
        -0.5902705704964453,
    ),
    phivar2 = (
```

```
        0.497835398462283,
        0.0008572932662572949,
        0.007043988949735419,
        0.01991030492217057,
        3.2482754058030023,
        2.2923143560178105,
        -3.3244075877556547,
    ),
);
```

In [11]:
```
ACTIVE_CASE = case1
cfg = build_run(ACTIVE_CASE)

sol0 = cfg.sol0
par = cfg.par
dd2 = par.d2
phivar1 = cfg.phivar1
phivar2 = cfg.phivar2

p_max = par.p_max
p_min = par.p_min

@show sol0;
@show phivar1;
@show phivar2;
@show par

#@show F_dhlen(sol0, par);
```

```
sol0 = [-4.25636992746322, -4.25636992746322, -6.564731755571115, -9.78495
9500018429, 3.4964061926987404, -4.881970605917915, -3.612611421864211, -
0.11480149900657971, -9.153701027345607, -0.02093818546842936, -0.97134019
12962828, -0.6969603658917966, -12.24792429700535, -11.622779834506675, 1
8.981276884494765, -5.514571709515935, 0.8472451549010297]
phivar1 = (0.0014072188552725978, 0.013975575489771453, 0.0139755754897714
53, 5.420054200542007e-5, 0, 1.9049897537435612)
phivar2 = (0.4713311048662077, 0.00752500303794388, 0.026272431023913592,
9.901500213468598e-5, 0.5717491375633551, 2.5747871316756075)
par = (d2 = 0.22102128118460218, chi = 0.85, N = 7, nuA = 7.14285714285714
3, lambda = 100.0, KM = 0.0013550135501355014, switch_dh = 1, zplus = 1.0,
zpep = [1, 1], p_max = 125.0, p_min = 0.1)
```

Out[11]:
```
(d2 = 0.22102128118460218, chi = 0.85, N = 7, nuA = 7.142857142857143, l
ambda = 100.0, KM = 0.0013550135501355014, switch_dh = 1, zplus = 1.0, z
pep = [1, 1], p_max = 125.0, p_min = 0.1)
```

In [12]:
```
# qdlen = plot()

# ev0(phiplus,phipep)=evmin(phiplus,phiminus/par.zplus,phipep)

phiplus_r = range(0.00001, 0.4 + 0.0 / (1 + par.zplus), length = 1000)
phipep_r = range(
    0.00001,
    0.5 + 0.0 / (1 + (par.zpep[1] + par.zpep[2]) / (par.N * par.nuA)),
    length = 1000,
)

Z = [evmin([x, x * par.zplus, y], par) for y in phipep_r, x in phiplus_r]

cp =
    contour(phiplus_r, phipep_r, Z; levels = [0], linewidth = 2, color =
```

```
# Zdet = [hessdet([x, x * par.zplus, y], par) for y in phipep_r, x in phi

cpdet = deepcopy(cp)

# contour!(
#     cpdet,
#     phiplus_r,
#     phipep_r,
#     Zdet;
#     levels = [0],
#     linewidth = 2,
#     color = :white,
#     linestyle = :dash,
#     cbar = false,
# )

display(cpdet)

# plot!(
#     qdlen,
#     br,
#     vars = (:param, :phiplus1),
#     branchlabel = ["br"],
#     legend = :topright,
#     color = :red,
# )
# display(qdlen)

# using DataFrames, CSV

# df = DataFrame(
#     phiplus = repeat(phiplus_r, outer = length(phipep_r)),
#     phipep = repeat(phipep_r, inner = length(phiplus_r)),
#     Z = vec(Z),
# )

# CSV.write("contour_data.csv", df)


# using Printf
# import Contour

# Zc = permutedims(Z)  # convert from rows=y, cols=x to rows=x, cols=y

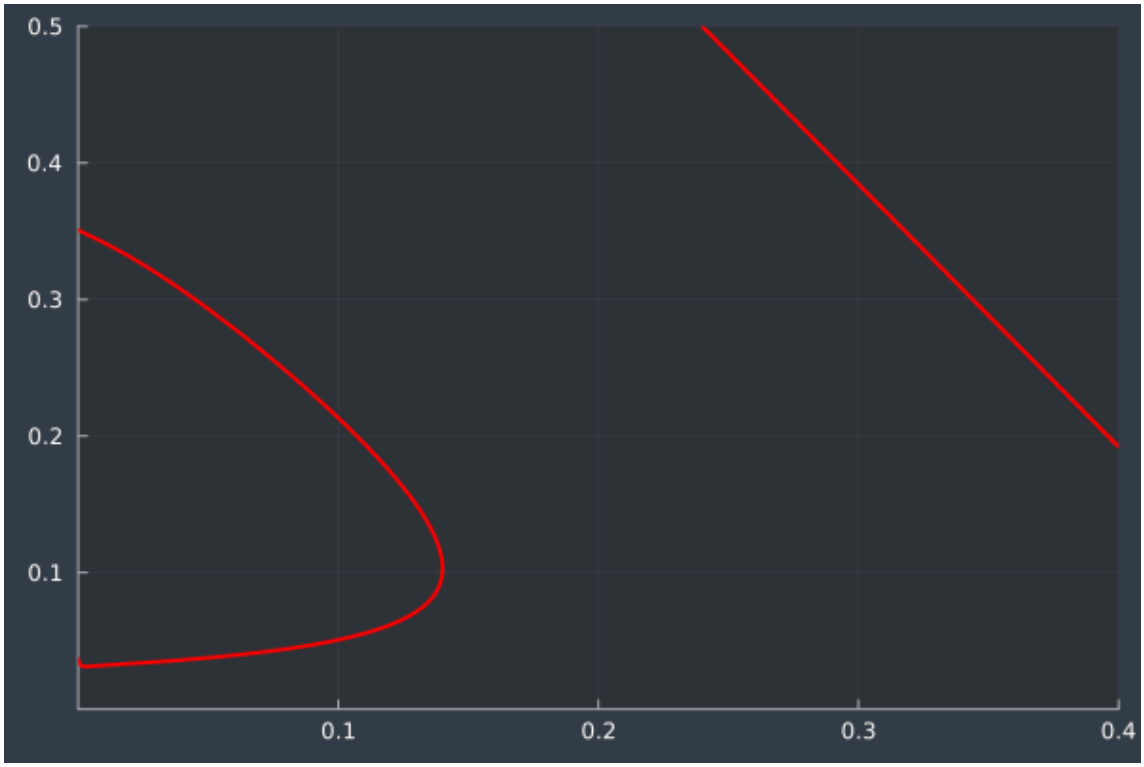
# open("contour0.dat", "w") do io
#     println(io, "# contour data")
#     println(io, "# columns: x y")
#     println(io, "# blank line separates different contour lines")

#     cs = Contour.contours(phiplus_r, phipep_r, Zc, [0.0])

#     seg = 0
#     for cl in Contour.levels(cs)
#         for ln in Contour.lines(cl)
#             seg += 1
#             xs, ys = Contour.coordinates(ln)

#             println(io, "# segment ", seg)
#             for (x, y) in zip(xs, ys)
```

```
#                 @printf(io, "%.16g %.16g\n", x, y)
#             end
#             println(io)  # empty line between contour lines
#         end
#     end
# end
```



In [13]:
```
probdlen = BK.BifurcationProblem(
    F_dhlen,
    sol0,
    par,
    (@optic _.lambda),
    record_from_solution = (x, p; kwargs...) -> (
        phiplus1 = tf(x[1]),
        phiminus1 = tf(x[2]),
        phipep1 = tf(x[3]),
        phioh1 = tf(x[4]),
        phiw1 = tf(x[5]),
        phiplus2 = tf(x[5+1]),
        phiminus2 = tf(x[5+2]),
        phipep2 = tf(x[5+3]),
        phioh2 = tf(x[5+4]),
        phiw2 = tf(x[5+5]),
        delphipm1 = tf(x[2]) - tf(x[1]),
        delphipm2 = tf(x[5+2]) - tf(x[5+1]),
    ),
)

#optnewton = NewtonPar(tol = 1e-8, verbose = true, linesearch = true, α =
optnewton = NewtonPar(tol = 1e-8, verbose = true, linesearch = true, α =
sol = BK.solve(probdlen, Newton(), optnewton)


println(
    "phiplus1 = $(tf(sol.u[1]))\n" *
    "phiminus1 = $(tf(sol.u[2]))\n" *
    "phipep1 = $(tf(sol.u[3]))\n" *
```

```
    "phioh1 = $(tf(sol.u[4]))\n" *
    "phiw1 = $(tf(sol.u[5]))\n" *
    "phiplus2 = $(tf(sol.u[5+1]))\n" *
    "phiminus2 = $(tf(sol.u[5+2]))\n" *
    "phipep2 = $(tf(sol.u[5+3]))\n" *
    "phioh2 = $(tf(sol.u[5+4]))\n" *
    "phiw2 = $(tf(sol.u[5+5]))\n",
)

println(
    "phivar1 = ( \n$(tf(sol.u[3])),\n" *
    "$(tf(sol.u[1])),\n" *
    "$(tf(sol.u[2])),\n" *
    "0,\n" *
    "0,\n" *
    "0,\n" *
    "),\n" *
    "phivar2 = ( \n$(tf(sol.u[3+5])),\n" *
    "$(tf(sol.u[1+5])),\n" *
    "$(tf(sol.u[2+5])),\n" *
    "0,\n" *
    "0,\n" *
    "0,\n" *
    "),\n",
)

@show sol.prob.params.lambda;
```

```
┌─────────────┬──────────────┬──────────────────┐
│ Newton step │     residual │ linear iterations │
├─────────────┼──────────────┼──────────────────┤
│           0 │    5.2205e-05 │                0 │
│           1 │    8.8761e-04 │                1 │
│           2 │    4.1038e-06 │                1 │
│           3 │    5.4892e-10 │                1 │
└─────────────┴──────────────┴──────────────────┘
```

```
phiplus1 = 0.012443572987289849
phiminus1 = 0.012443572987289849
phipep1 = 0.0013550135501355426
phioh1 = 5.420054200542178e-5
phiw1 = 0.9737036399453718
phiplus2 = 0.0066644734446563965
phiminus2 = 0.025464620411987642
phipep2 = 0.4727765845945261
phioh2 = 0.00011091639271464647
phiw2 = 0.4949834051858683

phivar1 = (
0.0013550135501355426,
0.012443572987289849,
0.012443572987289849,
0,
0,
0,
),
phivar2 = (
0.4727765845945261,
0.0066644734446563965,
0.025464620411987642,
0,
0,
0,
),

sol.prob.params.lambda = 100.0
```

In [14]:

```
optscontdlen = ContinuationPar(
    dsmin = 0.000001,
    dsmax = 0.01,
    ds = 0.01,
    #        p_max = dd2*1.053499, #6/(1+par.N*par.nuA/2),
    #        p_min = dd2*0.999, #0.00001,
    p_max = p_max,
    p_min = p_min,
    nev = 0,
    detect_fold = true,
    plot_every_step = 1,
    newton_options = NewtonPar(max_iterations = 500, tol = 1e-12, verbose
    max_steps = 1000000,
    detect_bifurcation = 0,
    tol_stability = 1e-14,
)

alg = PALC(tangent = Bordered())
br = @time continuation(
    probdlen,
    alg,
    optscontdlen;
```

```
    plot = false,
    verbosity = 0,
    bothside = true,
)
```

```
 11.372264 seconds (34.40 M allocations: 2.182 GiB, 9.96% gc time, 90.78%
compilation time)
```

Out[14]:

```
┌─ Curve type: EquilibriumCont
├─ Number of points: 5547
├─ Type of vectors: Vector{Float64}
├─ Parameter lambda starts at 125.0, ends at 125.0
├─ Algo: PALC [Bordered]
└─ Special points:

- #  1, endpoint at lambda ≈ +125.00000000,
step =   0
- #  2,     fold at lambda ≈ +85.99640672 ∈ (+85.98681045, +85.9868104
5), |δp|=-1e+00, [    guess], δ = ( 0,  0), step = 2782
- #  3, endpoint at lambda ≈ +125.00000000,
step = 5546
```

In [15]:

```
@show br
@show typeof(br)
br.branch[1]
```

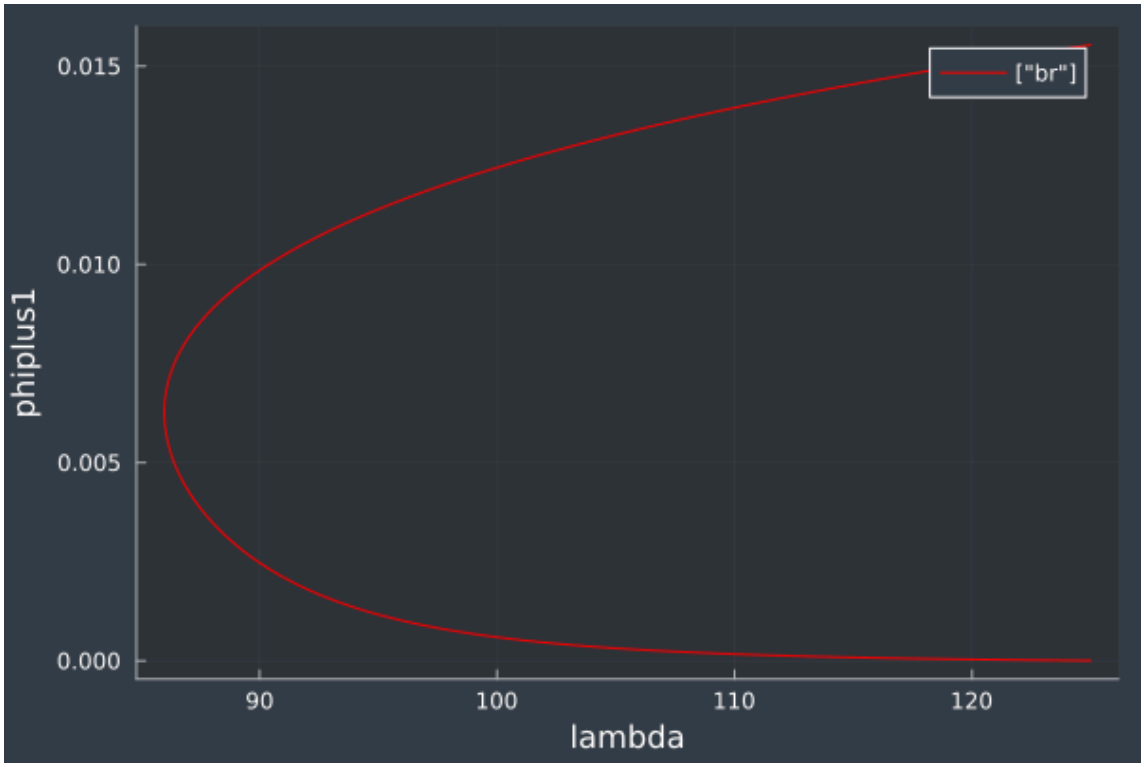
```
br =  ┌─ Curve type: EquilibriumCont
 ├─ Number of points: 5547
 ├─ Type of vectors: Vector{Float64}
 ├─ Parameter lambda starts at 125.0, ends at 125.0
 ├─ Algo: PALC [Bordered]
 └─ Special points:

- #  1, endpoint at lambda ≈ +125.00000000,
step =   0
- #  2,     fold at lambda ≈ +85.99640672 ∈ (+85.98681045, +85.98681045),
|δp|=-1e+00, [    guess], δ = ( 0,  0), step = 2782
- #  3, endpoint at lambda ≈ +125.00000000,
step = 5546

typeof(br) = ContResult{BifurcationKit.EquilibriumCont, @NamedTuple{phiplu
s1::Float64, phiminus1::Float64, phipep1::Float64, phioh1::Float64, phiw
1::Float64, phiplus2::Float64, phiminus2::Float64, phipep2::Float64, phioh
2::Float64, phiw2::Float64, delphipm1::Float64, delphipm2::Float64, para
m::Float64, itnewton::Int64, itlinear::Int64, ds::Float64, n_unstable::Int
64, n_imag::Int64, stable::Nothing, step::Int64}, Nothing, Nothing, Specia
lPoint{Float64, @NamedTuple{phiplus1::Float64, phiminus1::Float64, phipep
1::Float64, phioh1::Float64, phiw1::Float64, phiplus2::Float64, phiminus
2::Float64, phipep2::Float64, phioh2::Float64, phiw2::Float64, delphipm1::
Float64, delphipm2::Float64}, Vector{Float64}, Vector{Float64}}, Vector{@N
amedTuple{x::Vector{Float64}, p::Float64, step::Int64}}, ContinuationPar{F
loat64, DefaultLS, DefaultEig{typeof(real)}}, BifurcationProblem{BifFunctio
on{typeof(F_dhlen), BifurcationKit.var"#150#151"{typeof(F_dhlen)}, Nothin
g, Nothing, Nothing, Nothing, Nothing, Nothing, Nothing, Nothing, Nothing,
Float64, Nothing}, Vector{Float64}, @NamedTuple{d2::Float64, chi::Float64,
N::Int64, nuA::Float64, lambda::Float64, KM::Float64, switch_dh::Int64, zp
lus::Float64, zpep::Vector{Int64}, p_max::Float64, p_min::Float64}, Proper
tyLens{:lambda}, typeof(BifurcationKit.plot_default), var"#8#9", typeof(Bi
furcationKit.save_solution_default), typeof(BifurcationKit.update_defaul
t)}, PALC{Bordered, MatrixBLS{DefaultLS}, Float64, BifurcationKit.DotTheta
{BifurcationKit.var"#DotTheta##0#DotTheta##1", BifurcationKit.var"#DotThet
a##2#DotTheta##3"}}}
```

Out[15]:
```
(phiplus1 = 4.473035750763891e-6, phiminus1 = 4.473035750763891e-6, phip
ep1 = 0.0013550135501355016, phioh1 = 5.420054200542005e-5, phiw1 = 0.99
85818398363575, phiplus2 = 2.332346883079215e-6, phiminus2 = 0.001526141
9131390533, phipep2 = 0.5004083273940648, phioh2 = 0.018492523529506623,
phiw2 = 0.4795706748164065, delphipm1 = 0.0, delphipm2 = 0.0015238095662
55974, param = 125.0, itnewton = 3, itlinear = 3, ds = -0.01, n_unstable
= -1, n_imag = -1, stable = nothing, step = 0)
```

In [16]:
```
qdlen = plot()
plot!(
    qdlen,
    br,
    vars = (:param, :phiplus1),
    branchlabel = ["br"],
    legend = :topright,
    color = :red,
)
display(qdlen)
```



In [17]:
```
comp = 0
if (comp == 1)
    @load "mgcl2_violet_branch_data.jld2" data_pairs

    # Unpack for plotting
    params = first.(data_pairs)
    records = last.(data_pairs)

    # Plot
    plot!(
        qdlen,
        params,
        records;
        xlabel = "parameter",
        ylabel = "record",
        legend = false,
        color = :green,
        linestyle = :dash,
    )
end
```

In [18]:
```
qdlen_phiplus1 = plot()
qdlen_phiminus1 = plot()
qdlen_phipep1 = plot()
qdlen_phioh1 = plot()
qdlen_phiw1 = plot()

plot!(
    qdlen_phiplus1,
    br,
    vars = (:param, :phiminus1),
    branchlabel = ["phi-"],
    legend = :topright,
    color = :red,
)
```

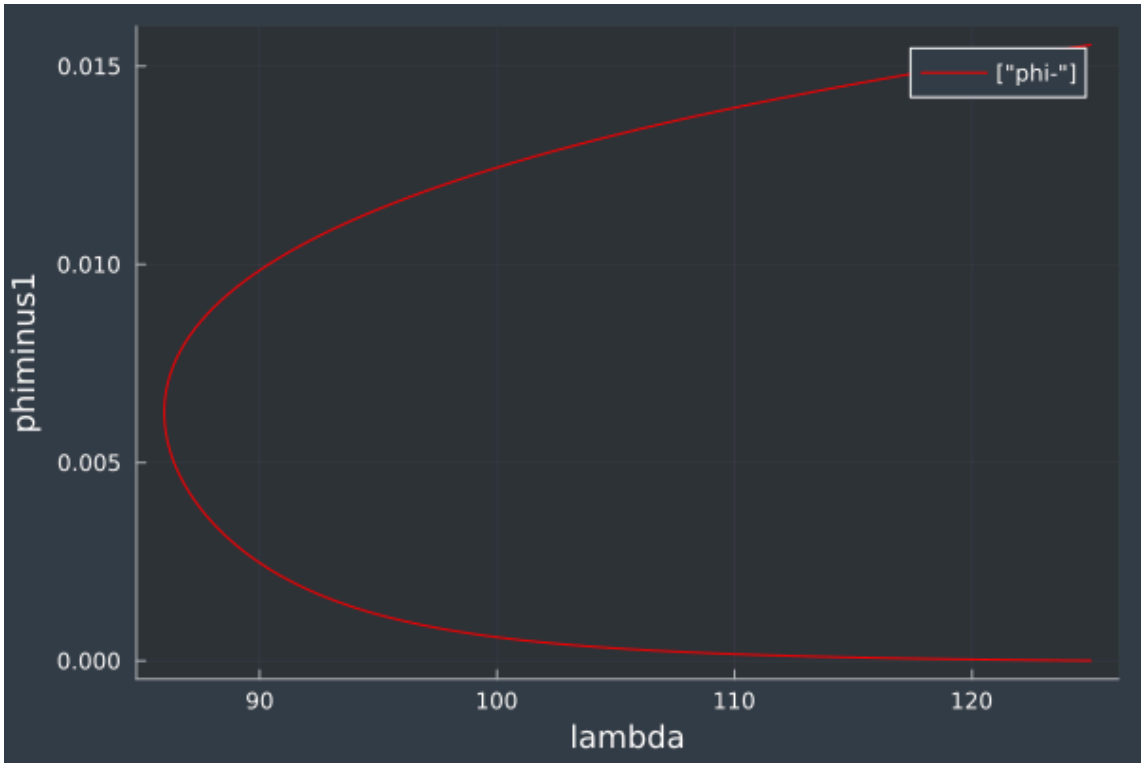
```
plot!(
    qdlen_phiminus1,
    br,
    vars = (:param, :phiplus1),
    branchlabel = ["phi+"],
    legend = :topright,
    color = :green,
)

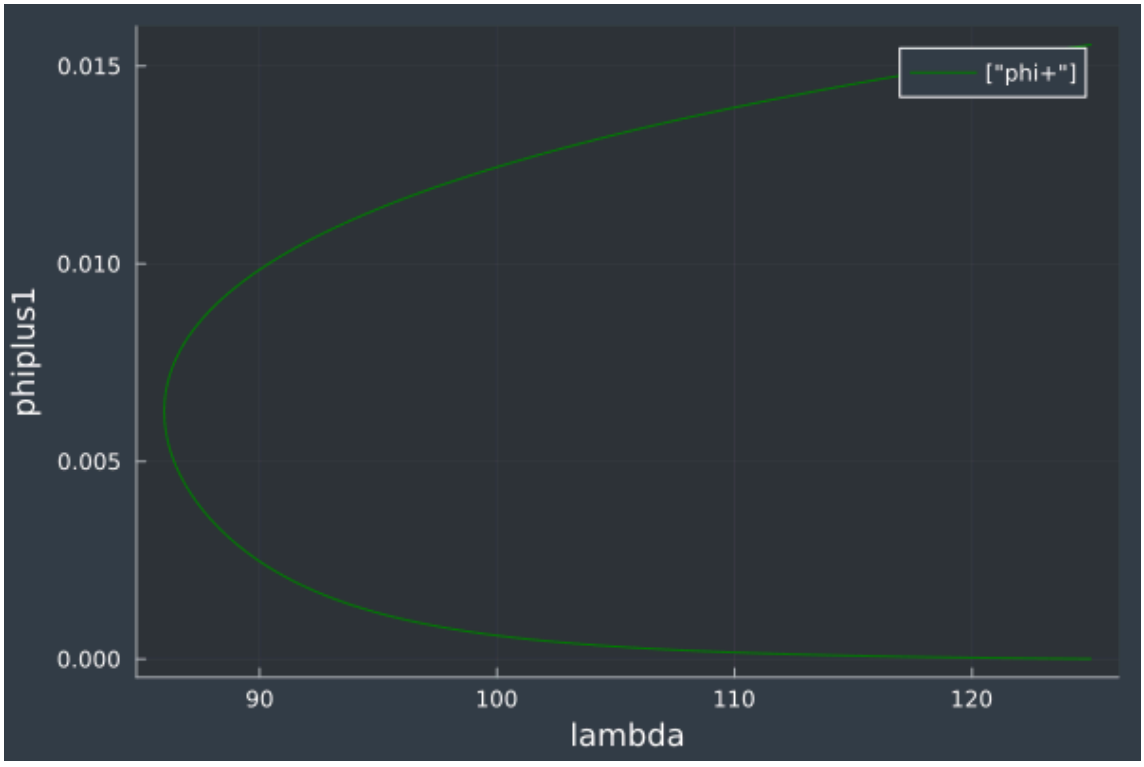
plot!(
    qdlen_phipep1,
    br,
    vars = (:param, :phipep1),
    branchlabel = ["peptide"],
    legend = :topright,
    color = :pink,
)

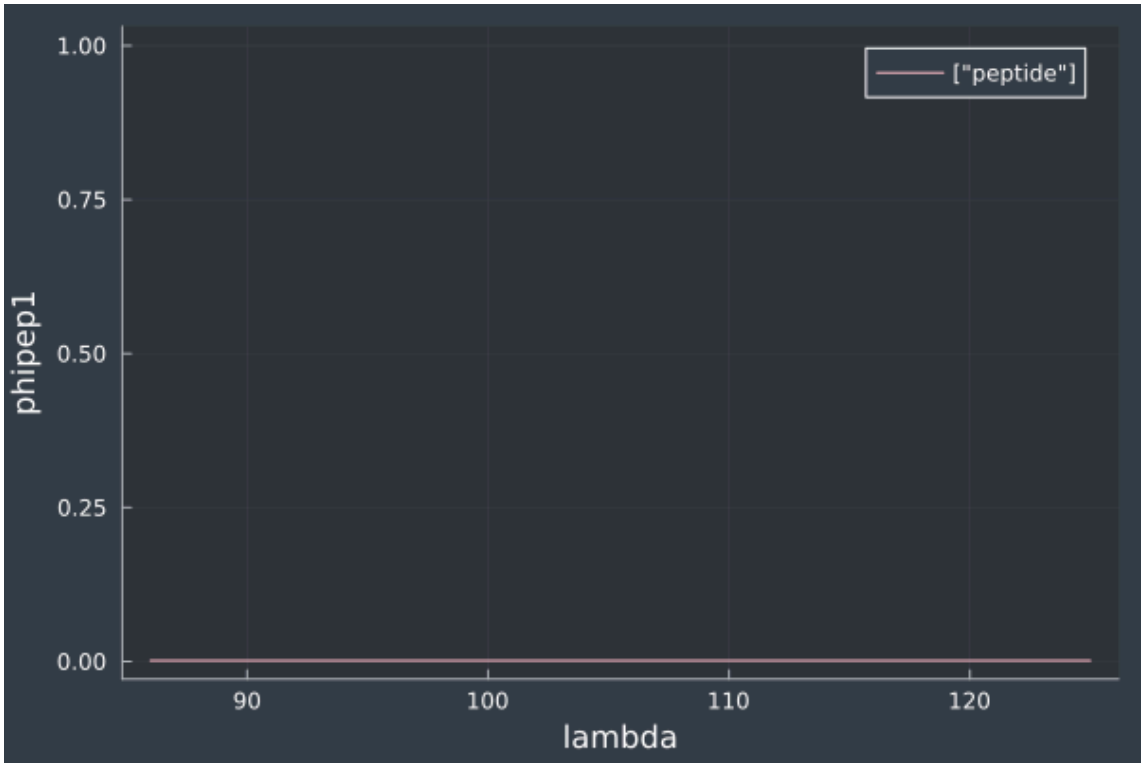
plot!(
    qdlen_phioh1,
    br,
    vars = (:param, :phioh1),
    branchlabel = ["OH-"],
    legend = :topright,
    color = :yellow,
)

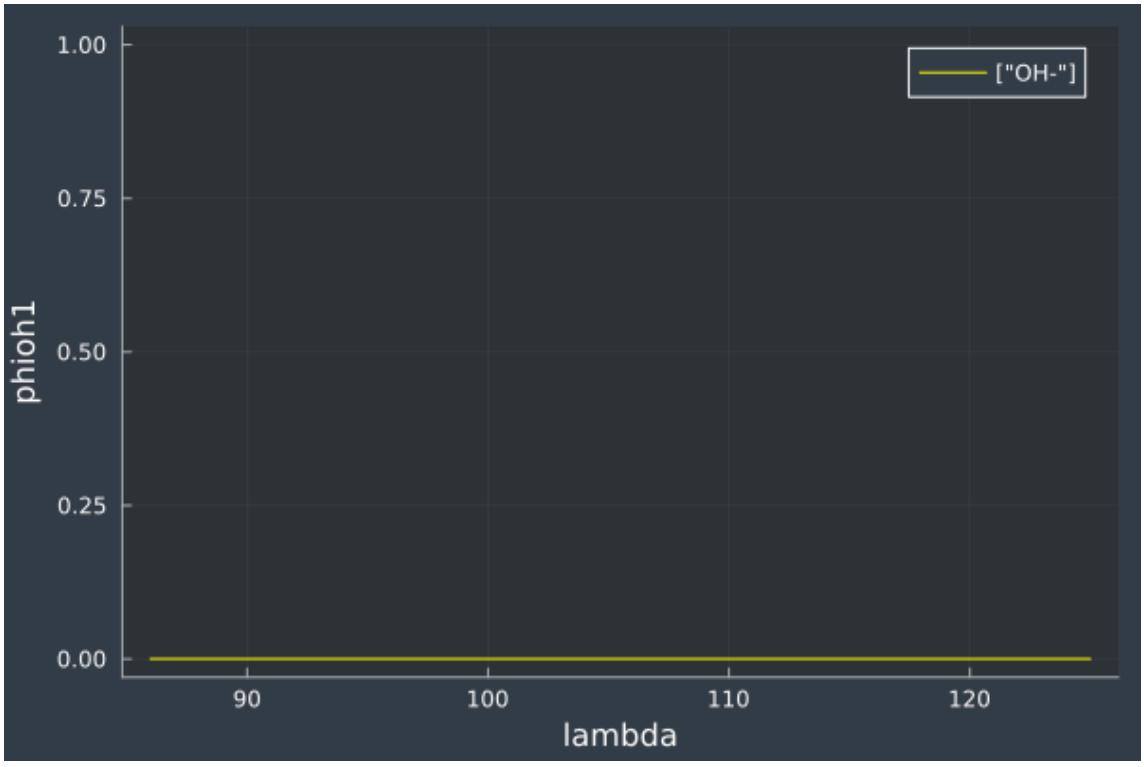
plot!(
    qdlen_phiw1,
    br,
    vars = (:param, :phiw1),
    branchlabel = ["water"],
    legend = :topright,
    color = :lightblue,
)

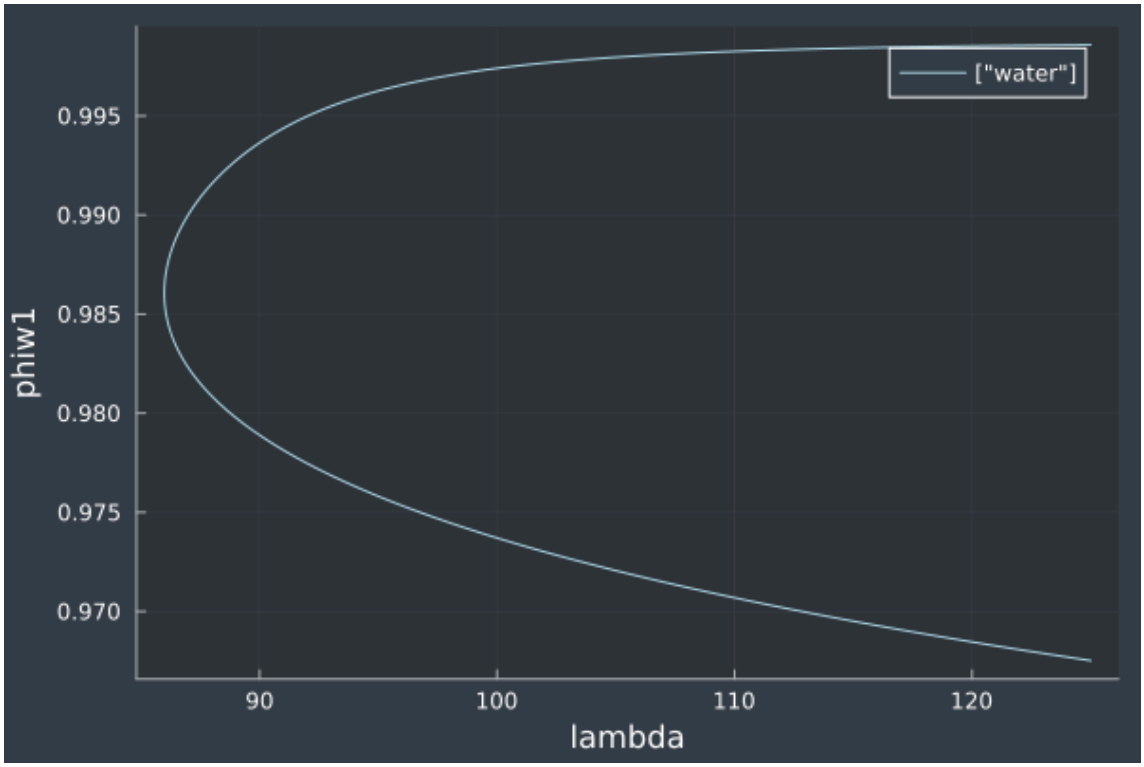
display(qdlen_phiplus1)
display(qdlen_phiminus1)
display(qdlen_phipep1)
display(qdlen_phioh1)
display(qdlen_phiw1)
```


0.015
0.010
0.005
0.000
phiminus1
90
100
110
120
lambda
["phi-"]


0.015
0.010
0.005
0.000
phiplus1
90
100
110
120
lambda
["phi+"]


1.00
0.75
0.50
0.25
0.00
phipep1
["peptide"]
90
100
110
120
lambda


1.00
0.75
0.50
0.25
0.00
phioh1
["OH-"]
90
100
110
120
lambda



In [19]:

```
qdlen_phiplus2 = plot()
qdlen_phiminus2 = plot()
qdlen_phipep2 = plot()
qdlen_phioh2 = plot()
qdlen_phiw2 = plot()

plot!(
    qdlen_phiplus2,
    br,
    vars = (:param, :phiminus2),
    branchlabel = ["phi-"],
    legend = :topright,
    color = :red,
)

plot!(
    qdlen_phiminus2,
    br,
    vars = (:param, :phiplus2),
    branchlabel = ["phi+"],
    legend = :topright,
    color = :green,
)

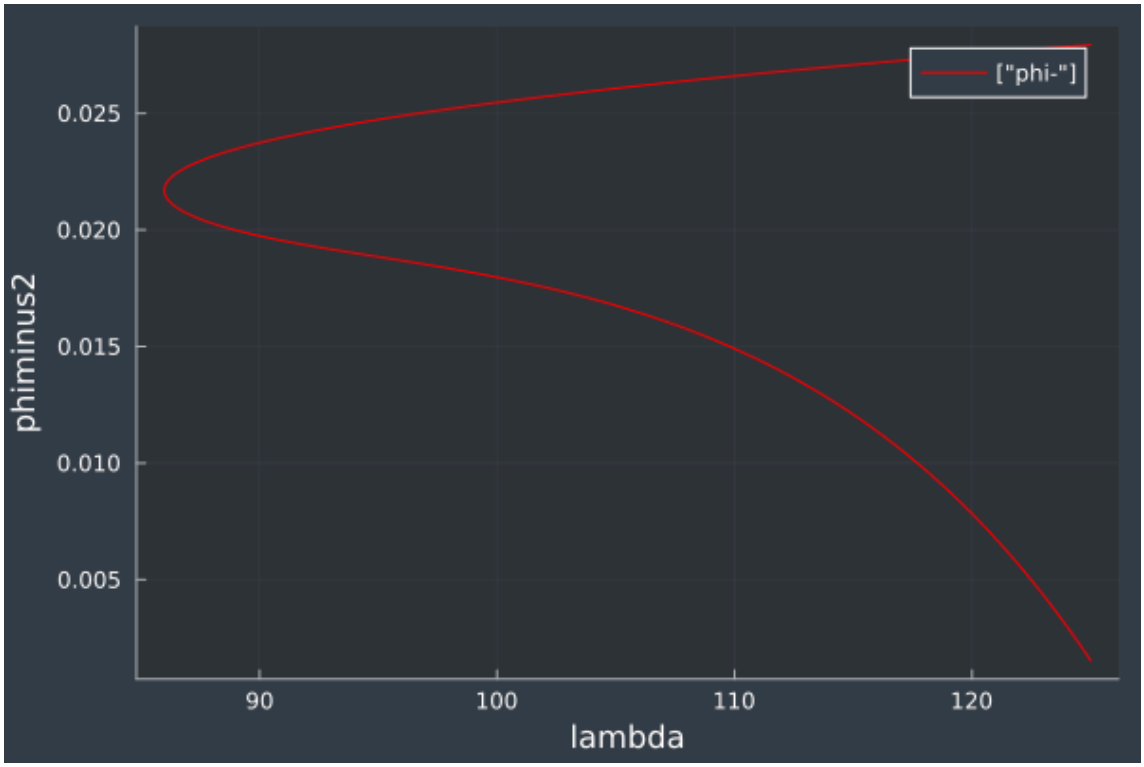
plot!(
    qdlen_phipep2,
    br,
    vars = (:param, :phipep2),
    branchlabel = ["peptide"],
    legend = :topright,
    color = :pink,
)

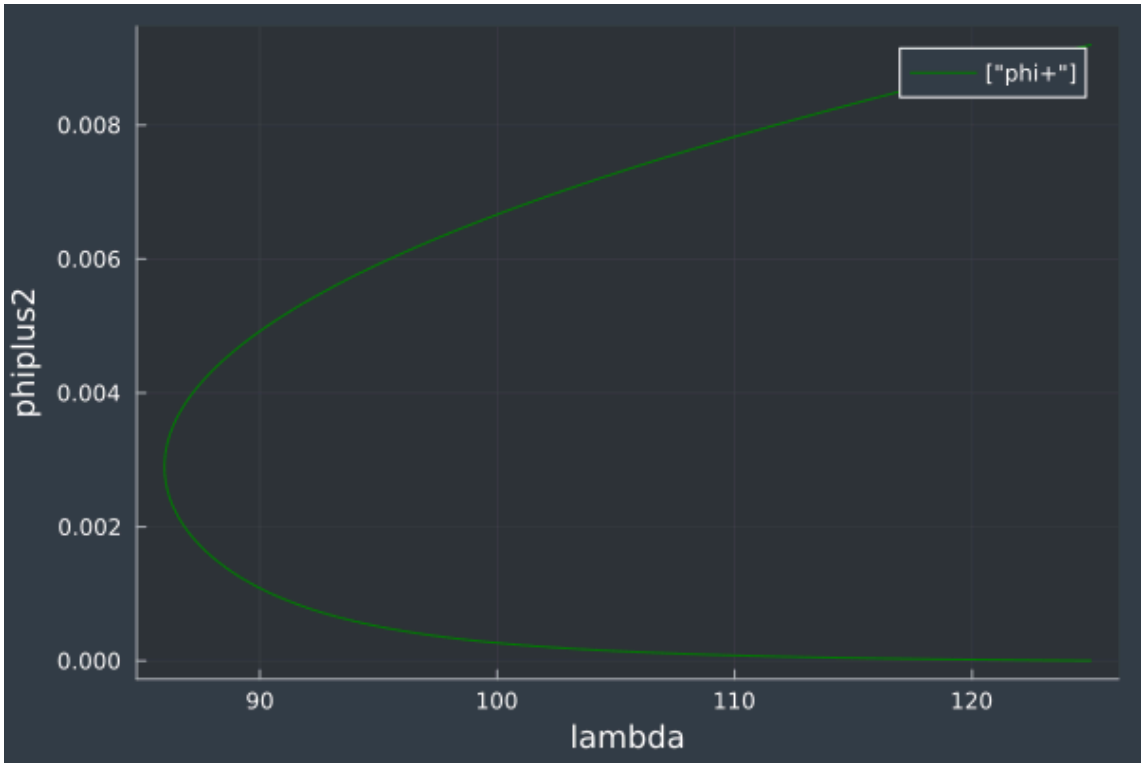
plot!(
    qdlen_phioh2,
    br,
```

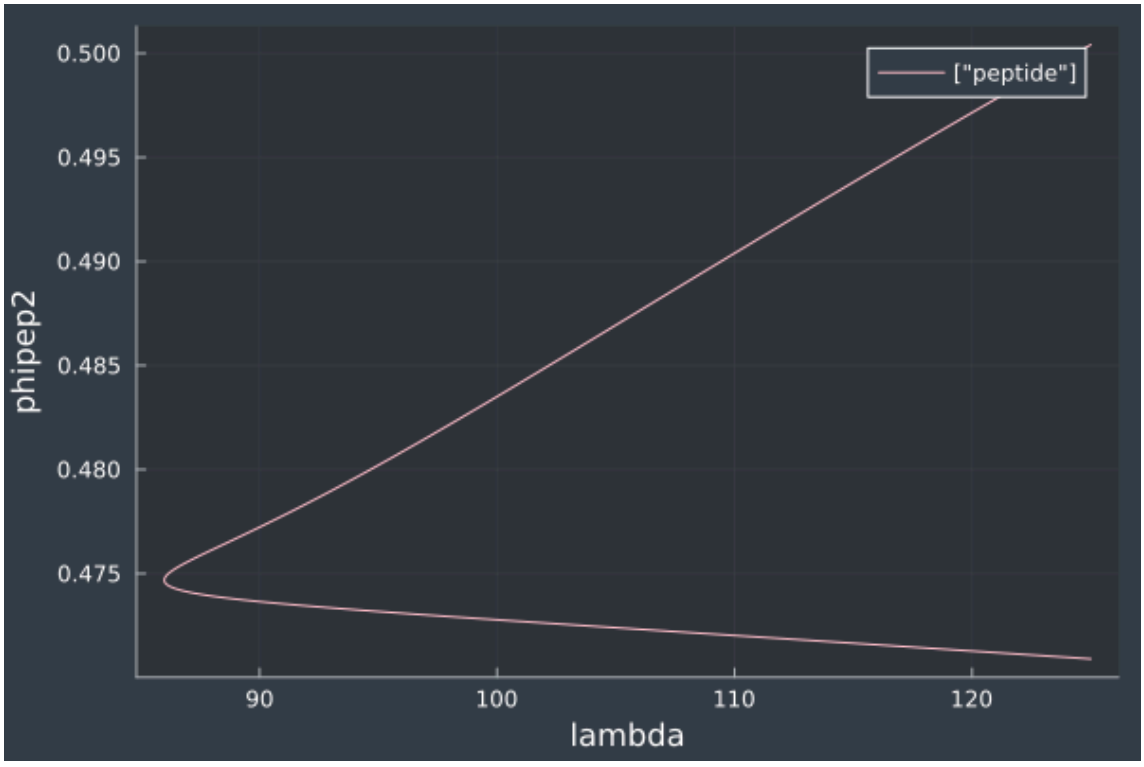
```
    vars = (:param, :phioh2),
    branchlabel = ["OH-"],
    legend = :topright,
    color = :yellow,
)

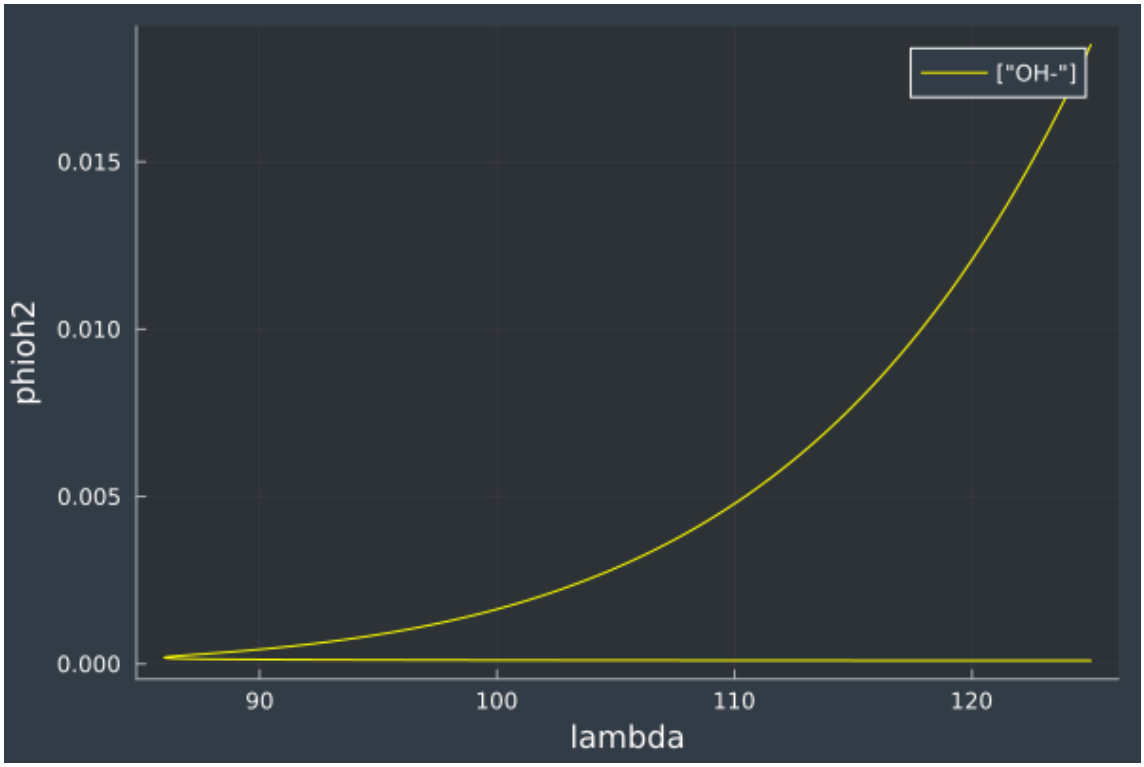
plot!(
    qdlen_phiw2,
    br,
    vars = (:param, :phiw2),
    branchlabel = ["water"],
    legend = :topright,
    color = :lightblue,
)

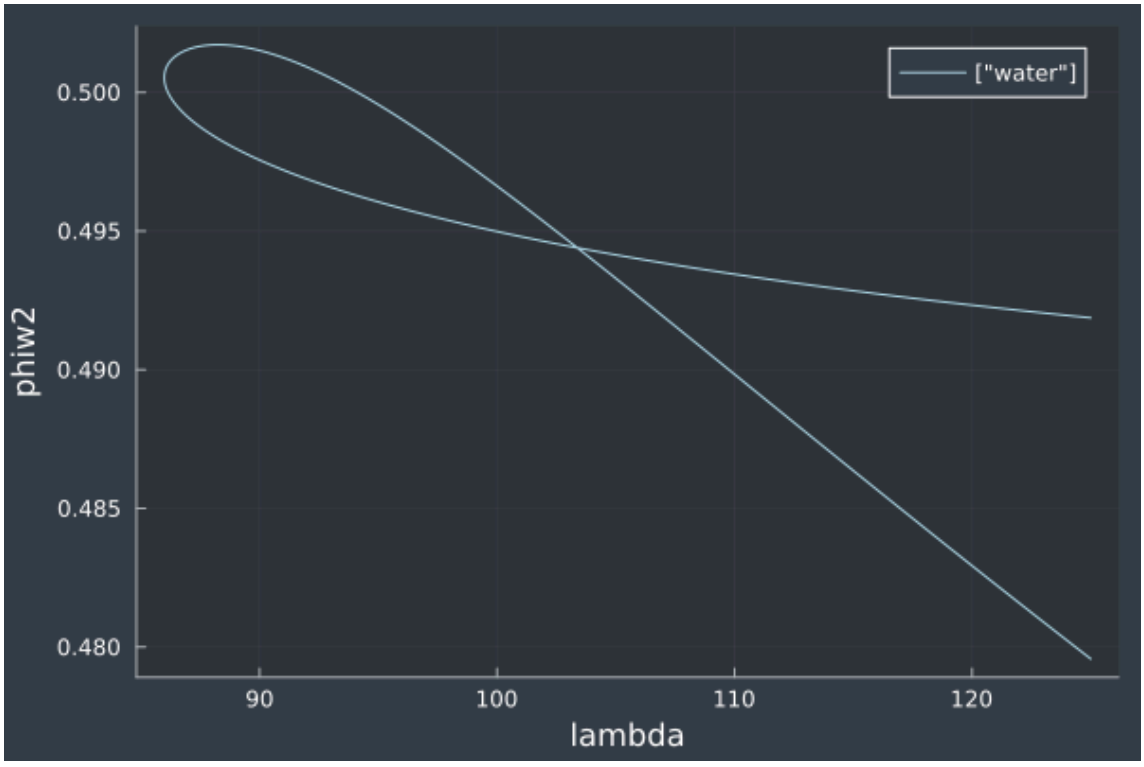
display(qdlen_phiplus2)
display(qdlen_phiminus2)
display(qdlen_phipep2)
display(qdlen_phioh2)
display(qdlen_phiw2)
```




["phi+"]
0.008
0.006
0.004
0.002
0.000
phiplus2
90
100
110
120
lambda


["peptide"]
0.500
0.495
0.490
0.485
0.480
0.475
phipep2
90
100
110
120
lambda





In [20]:
```
#br.phiw2
typeof(optscontdlen)
fieldnames(typeof(optscontdlen))
```

Out[20]:
```
(:dsmin, :dsmax, :ds, :a, :p_min, :p_max, :max_steps, :newton_options, :
η, :save_to_file, :save_sol_every_step, :nev, :save_eig_every_step, :sav
e_eigenvectors, :plot_every_step, :tol_stability, :detect_fold, :detect_
bifurcation, :dsmin_bisection, :n_inversion, :max_bisection_steps, :tol_
bisection_eigenvalue, :detect_event, :tol_param_bisection_event, :detect
_loop)
```

In [21]:
```
optscontdlen2 = ContinuationPar(
    dsmin = 0.0000001,
    dsmax = 0.00001,
```

```
    ds = 0.000001,
    #        p_max = dd2*1.053499, #6/(1+par.N*par.nuA/2),
    #        p_min = dd2*0.999, #0.00001,
    p_max = par.KM * 500,
    p_min = par.KM * 0.5,
    nev = 0,
    detect_fold = true,
    plot_every_step = 100,
    newton_options = NewtonPar(max_iterations = 500, tol = 1e-12, verbose
    max_steps = 1000000,
    detect_bifurcation = 0,
    tol_stability = 1e-14,
)


#optscontdlen2 = @set @set(optscontdlen.p_min = 1e-8).p_max = 1-1e-8


probdlen2 = BK.re_make(probdlen; lens = (@optic _.KM), u0 = sol.u)

@show probdlen2

@show probdlen2.u0
@show probdlen2.params

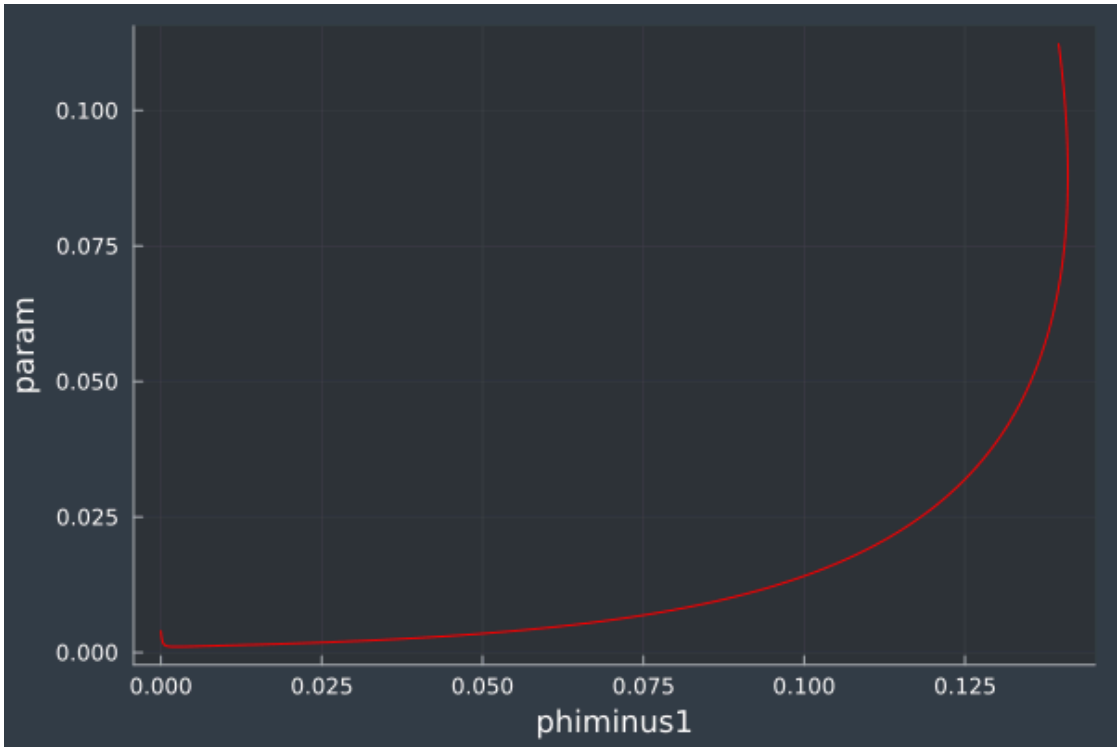
br = @time continuation(
    probdlen2,
    alg,
    optscontdlen2;
    plot = false,
    verbosity = 0,
    bothside = true,
);
```

```
probdlen2 = ┌─ Bifurcation problem with uType Vector{Float64}
├─ Inplace: false
├─ Dimension: 17
├─ Symmetric: false
└─ Parameter: KM
probdlen2.u0 = [-4.374029372721722, -4.374029372721722, -6.60258789218930
6, -9.822765447457734, 3.6116764575986617, -5.004277551313061, -3.64467076
9702184, -0.10900145695956766, -9.10662293670631, -0.020067052623845637, -
0.9745034688572713, -0.6986290550540388, -12.23459396590172, -11.564344412
66182, 19.032919297570945, -5.4362686342324915, 0.8458088438684455]
probdlen2.params = (d2 = 0.22102128118460218, chi = 0.85, N = 7, nuA = 7.1
42857142857143, lambda = 100.0, KM = 0.0013550135501355014, switch_dh = 1,
zplus = 1.0, zpep = [1, 1], p_max = 125.0, p_min = 0.1)
173.688224 seconds (1.86 G allocations: 246.459 GiB, 21.36% gc time, 0.81%
compilation time)
```

In [22]:

```
phiplot = plot(
    br,
    vars = (:phiminus1, :param),
    #    branchlabel = ["br"],
    legend = :topright,
    color = :red,
)
```

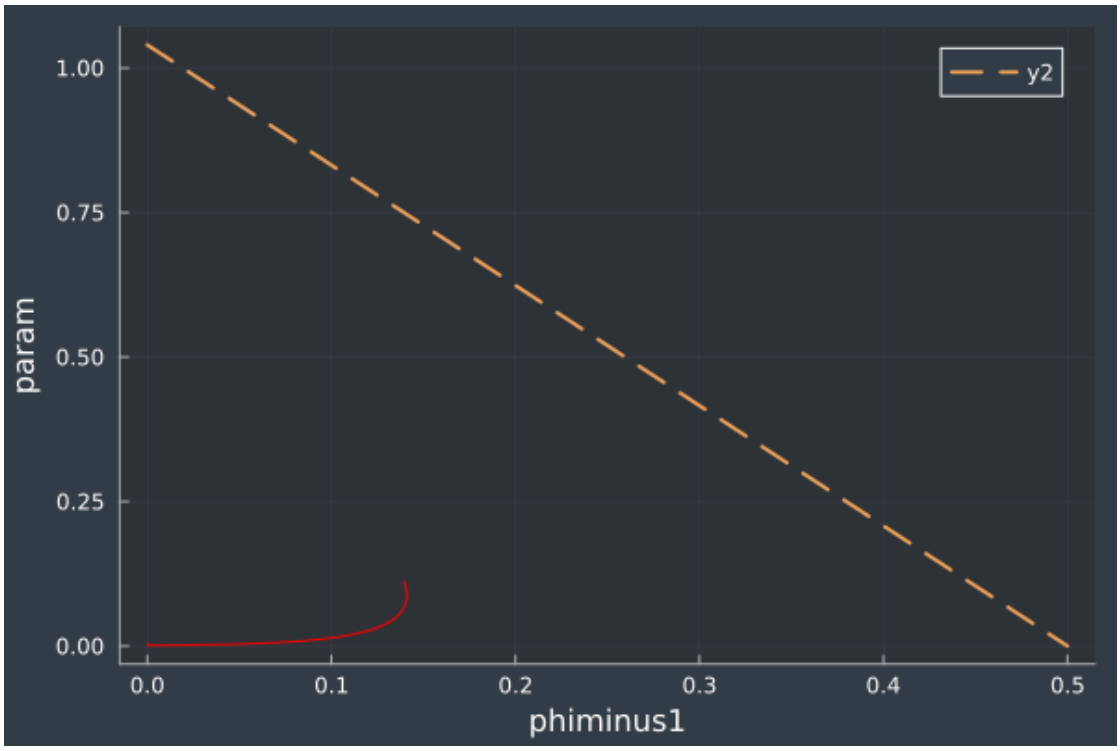
Out[22]:



In [23]:

```
xx = 0:(1/(par.zplus+1))/10:(1/(par.zplus+1))
yy = (1 .- (par.zplus + 1) .* xx) .* (1 + (par.zpep[1] + par.zpep[2]) / (
plot!(phiplot, xx, yy, linewidth = 2, linestyle = :dash)
# collect(xx)
```

Out[23]:

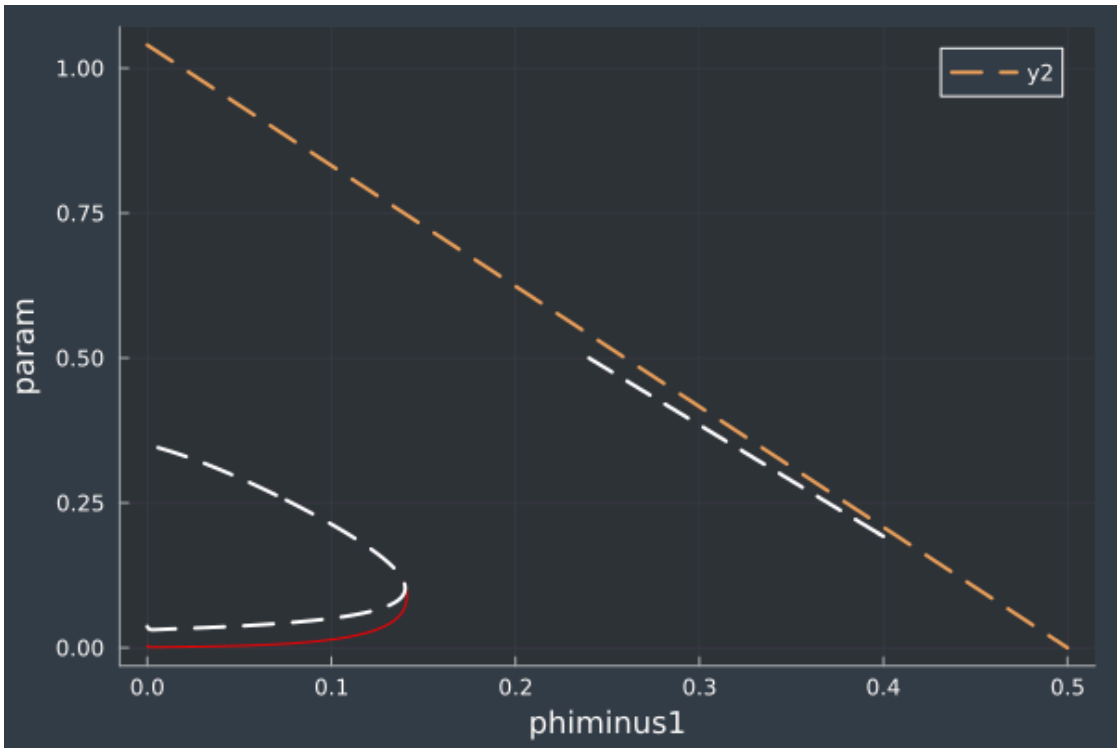


In [24]:

```
contour(
    phiplot,
    phiplus_r,
    phipep_r,
    Z;
    levels = [0],
    linewidth = 2,
    color = :white,
    linestyle = :dash,
```

```
    cbar = false,
)
```

Out[24]:



In [25]:

```
using Printf
import Contour

Zc = permutedims(Z)  # convert from rows=y, cols=x to rows=x, cols=y

open("contour0-souradip-vary-peptide.dat", "w") do io
    println(io, "# contour data")
    println(io, "# columns: x y")
    println(io, "# blank line separates different contour lines")

    cs = Contour.contours(phiplus_r, phipep_r, Zc, [0.0])

    seg = 0
    for cl in Contour.levels(cs)
        for ln in Contour.lines(cl)
            seg += 1
            xs, ys = Contour.coordinates(ln)

            println(io, "# segment ", seg)
            for (x, y) in zip(xs, ys)
                @printf(io, "%.16g %.16g\n", x, y)
            end
            println(io)  # empty line between contour lines
        end
    end
end
```

In [26]:

```
params = br.branch.param
records1 = br.branch.phiplus1
records2 = br.branch.phipep1

# Pair them up in a storeable format
#data_pairs = collect(zip(params, records1, records2));   # Vector{Tuple{
```

```
# Save
#@save "mgcl2_violet_branch_data.jld2" data_pairs;

# Form a 3-column matrix
table = hcat(records1, records2)

# Write as whitespace-separated text
writedlm("binodal-souradip-vary-peptide.dat", table)
```

In [ ]: